Inteligencia Artificial

ENRIQUE PELÁEZ Ph.D.

Escuela Superior Politécnica del Litoral – ESPOL

Facultad de Ingeniería en Electricidad y Computación – FIEC

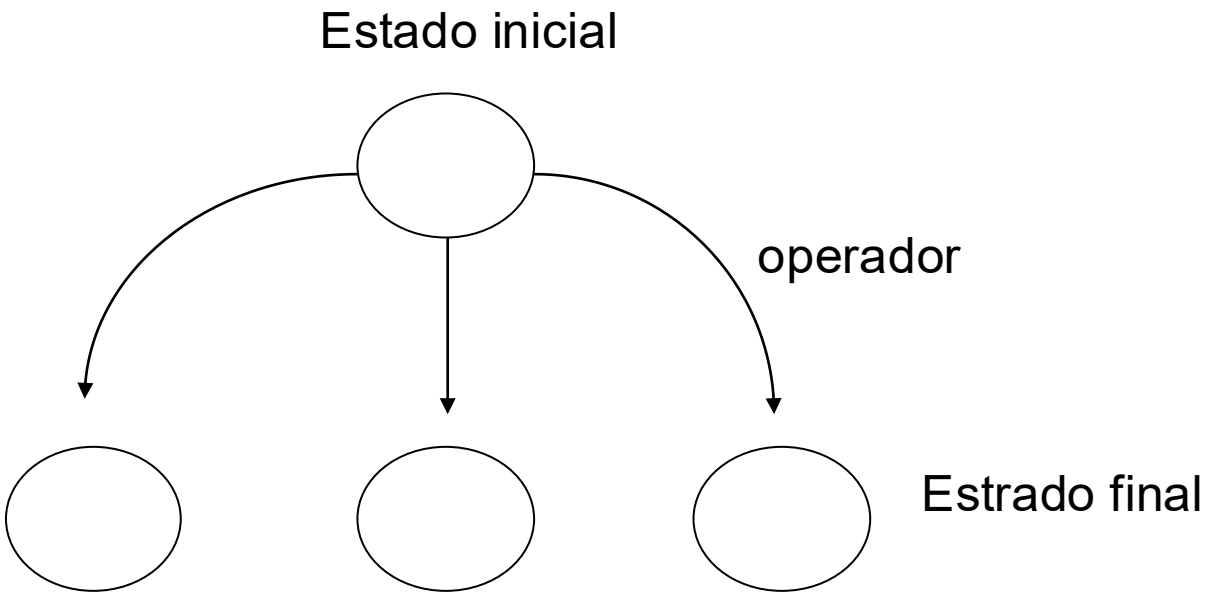
Clase # 22 - PAO 2026 – I

Agenda

1. Resolución de problemas basada en búsqueda
2. Búsqueda a ciegas
   2.1 Breath Search
   2.2 Depth Search
   2.3 Análisis de eficiencia: Convergencia, tiempo y memoria

Técnicas para resolver problemas en IA

- Búsqueda de estados



Técnicas para resolver problemas en IA

- Estrategias de búsqueda en el espacio de posibles estados
  - Goal-driven - encadenamiento hacia atrás
    - Toma el objetivo a ser resuelto. Ve qué reglas o movimientos legales podrían usarse para generar este objetivo y determina qué condiciones deben ser ciertas para llegar a este objetivo.
    - Estas nuevas condiciones se convierten en los nuevos objetivos para la búsqueda, se remonta a los hechos del problema.
    - Esto encuentra la cadena de movimientos o reglas que conducen de los datos a la meta, pero al revés.

Por ejemplo: Similar a: IF ….. THEN ……

Técnicas para resolver problemas en IA

- Data-driven: encadenamiento hacia adelante
  - La búsqueda comienza con los hechos dados del problema y un conjunto de movimientos legales o reglas para cambiar el estado.
  - La búsqueda continúa aplicando reglas a los hechos para producir nuevos hechos, hasta que genera un camino que satisface la condición del objetivo.

Técnicas para resolver problemas en IA

- Estrategias de búsqueda a ciegas:
    - Generate and Test
    - Breadth First
    - Depth First
    - Uniform cost search
    - Limited depth
    - Interactive depth
    - Bidirectional search

Generate and Test

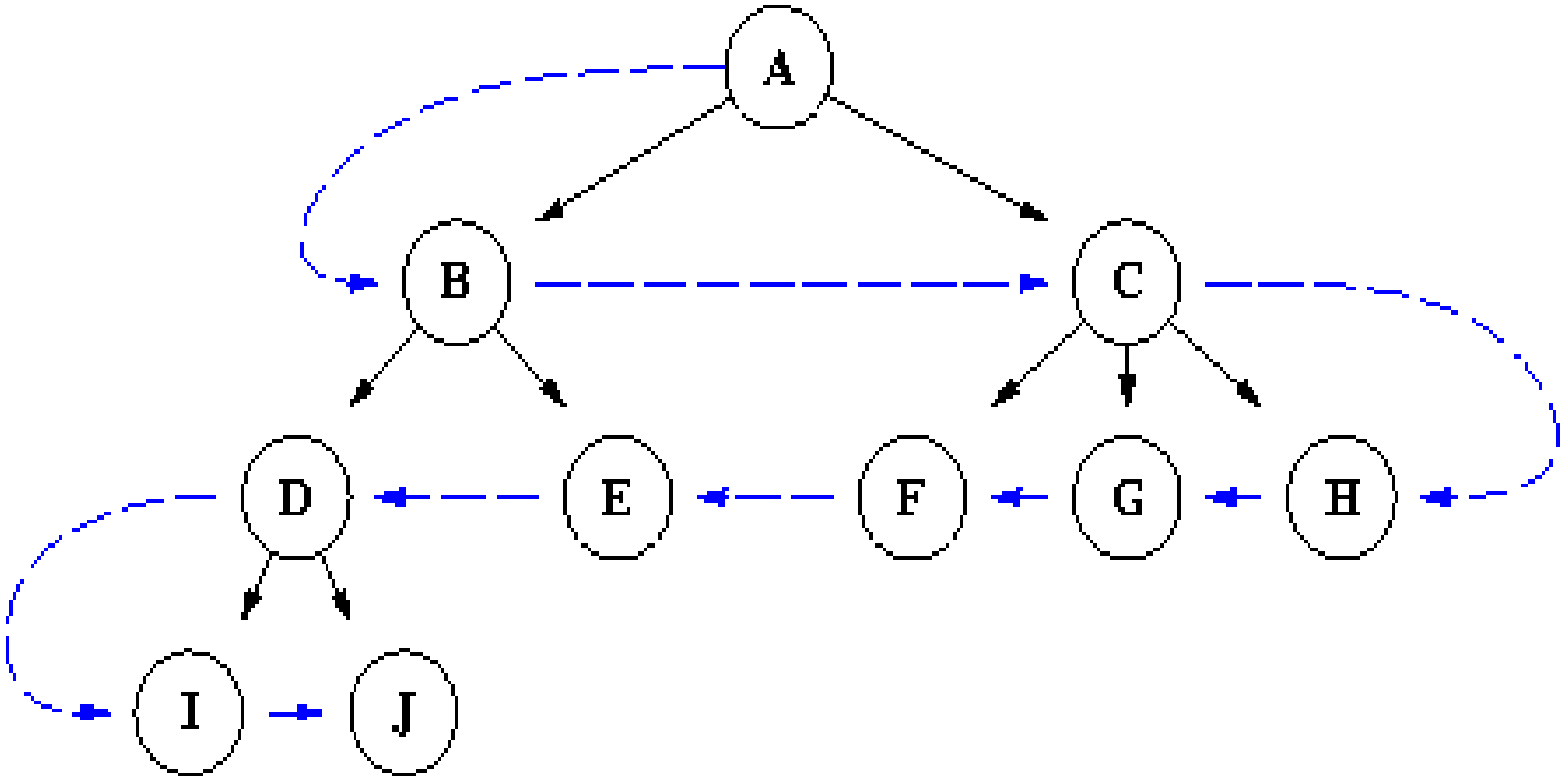
- Algoritmo

1. Genera una posible solución (estado o ruta)
2. Prueba para ver si es la solución. Compara con los elementos de un conjunto de soluciones aceptables.
3. Si se ha encontrado una solución, regrese. Si no, vaya al paso 1.

Generate and Test

- Una mejora del algoritmo es garantizar que una solución no se pruebe más de una vez. Esto se puede hacer enumerando las soluciones y probándolas en orden.
- Si las soluciones no se pueden enumerar, genere las soluciones al azar, pero mantenga una lista o una matriz con todas las soluciones probadas, para buscar una solución potencial antes de probarla.
- Esta alternativa tiene algunos problemas. Puede llevar tiempo buscar una solución probada, cuando la lista crece.
- Si el conjunto de posibles soluciones es grande, una mejor opción es generar aleatoriamente una solución sin probar.

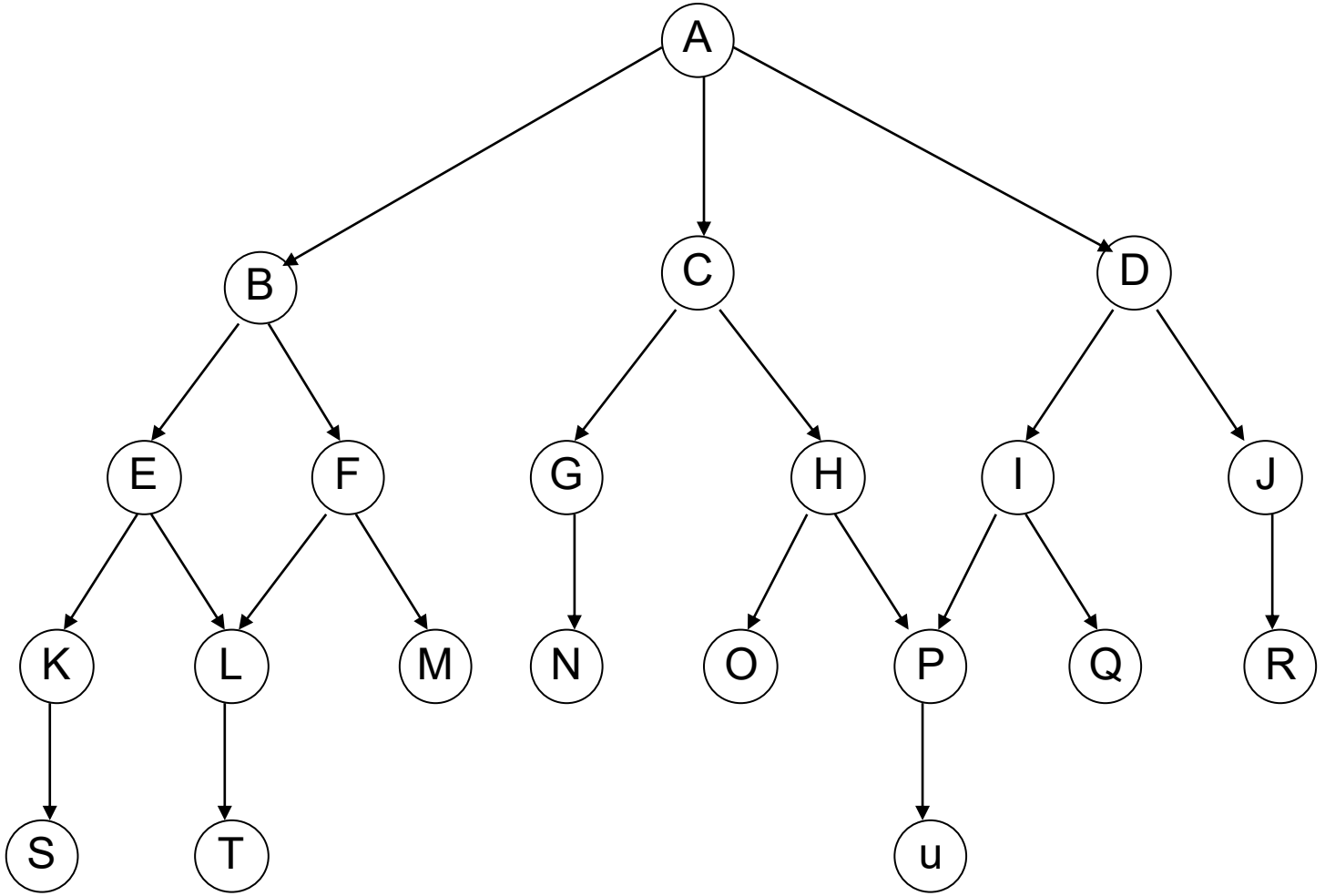
Breath - First



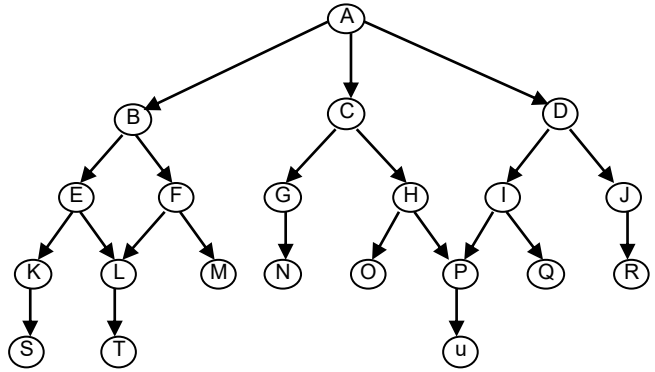
Breath - First

```
Begin
    open := [Start];
    closed := [ ];
    while open π [ ] do
        begin
            remove leftmost state from open, call it X;
                if X is a goal then returns SUCCESS
                    else begin
                        generate children of X;
                        put X on closed;
                        discard children of X if already on open or closed;
                        put remaining children on right end of open
                    end
        end
    return FAIL
end
```

Breath - First



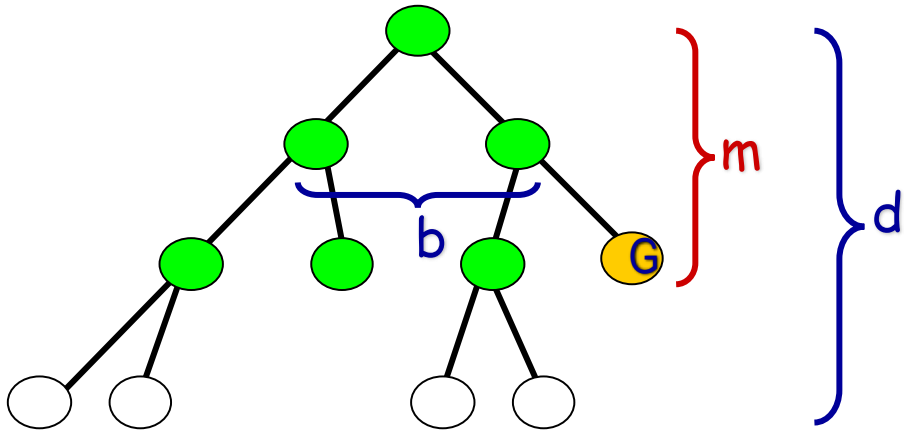
Breath - First



1. open = [A]; closed = [ ]
2. open = [B,C,D]; closed = [A]
3. open = [C,D,E,F]; closed = [B,A]
4. open = [D,E,F,G,H]; closed = [C,B,A]
5. open = [E,F,G,H,I,J]; closed = [D,C,B,A]
6. open = [F,G,H,I,J,K,L]; closed = [E,D,C,B,A]
7. open = [G,H,I,J,K,L,M] L is in open; closed = [F,E,D,C,B,A]
8. open = [H,I,J,K,L,M,N]; closed = [G,F,E,D,C,B,A]
9. Continue until U is found, or open = [ ].

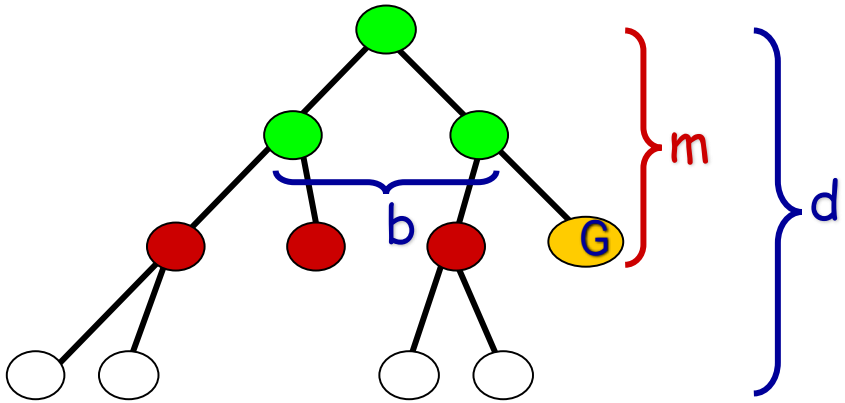
Breath - First

- Completo
  - ✓Si, (inclusive para arboles infinitos)
  - ✓Siempre encuentra la ruta más corta
- Tiempo
  - ✓$O(b^m)$

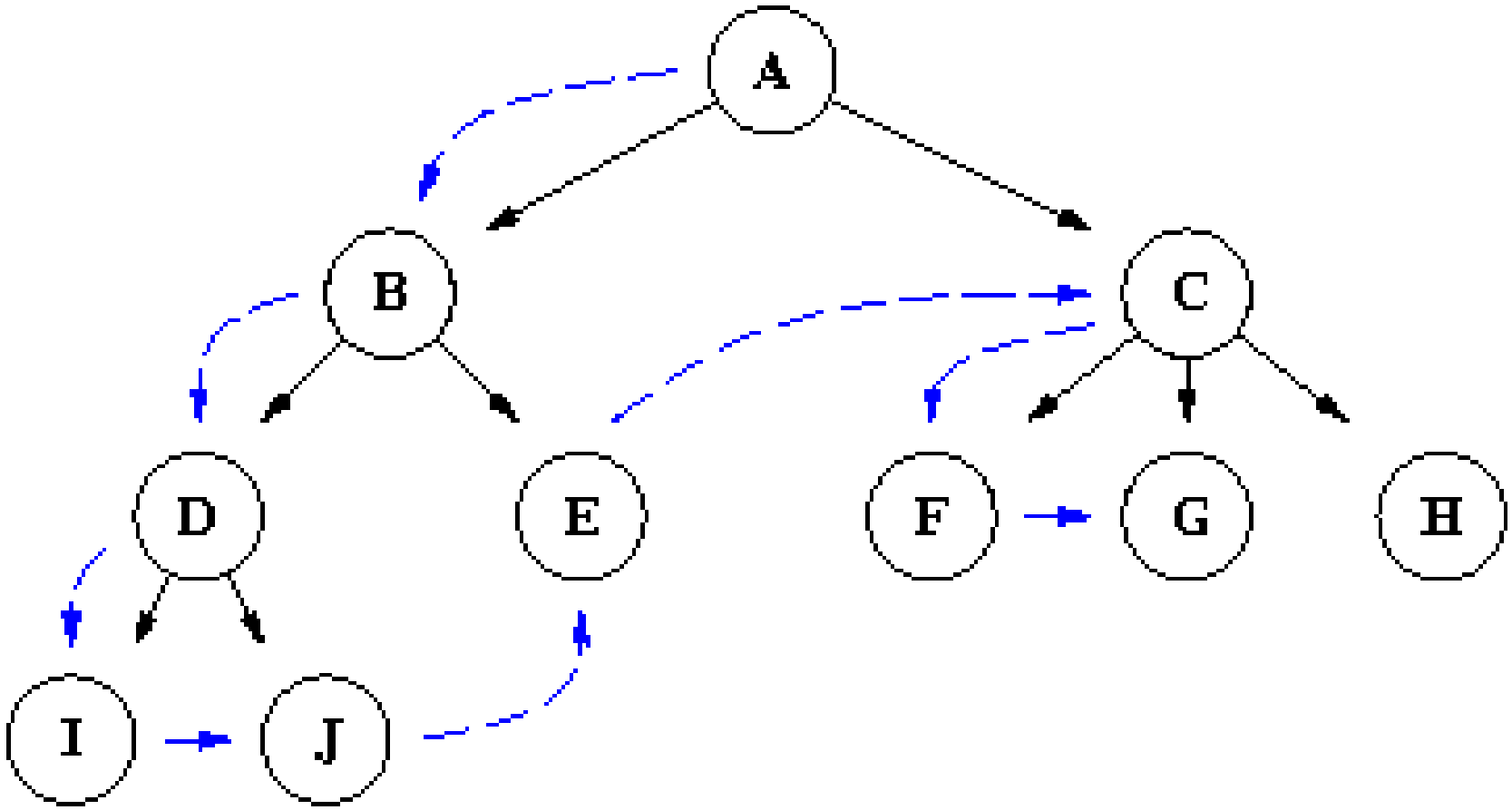


Breath - First

- Memoria
  - El número máximo de nodos se alcanza en el nivel "m" (objetivo)
  - En general: b^m
  - Generalmente "peor" que la profundidad primero



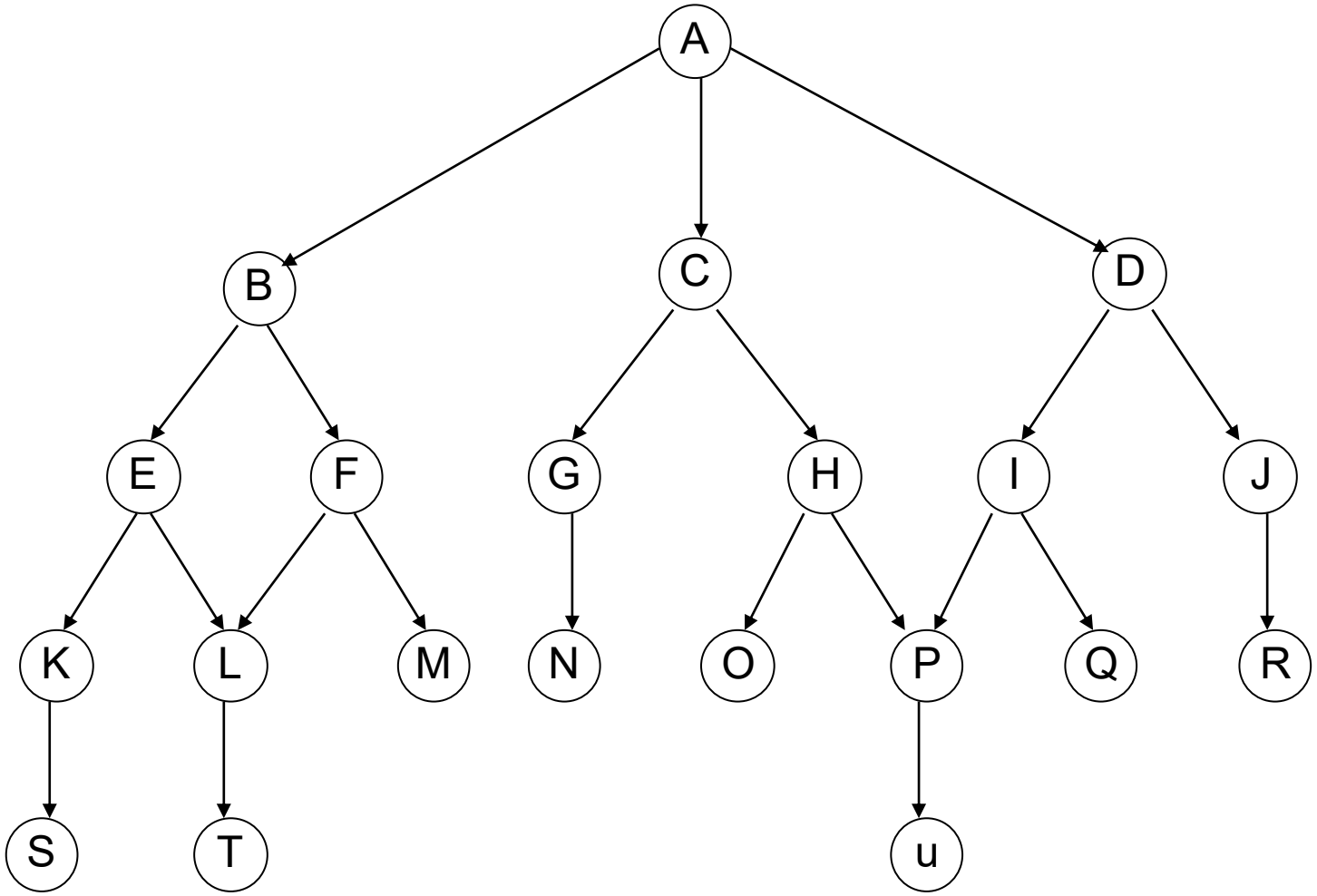
Depth - First



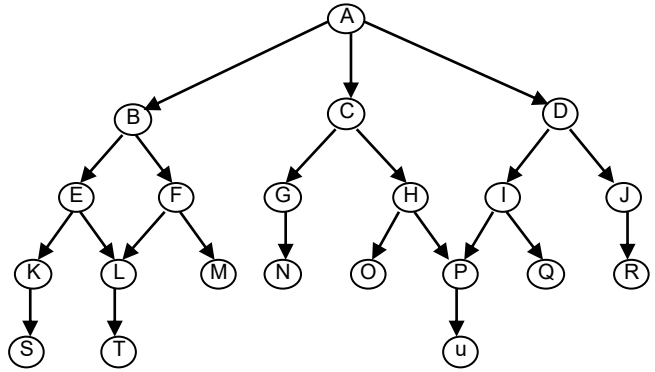
Depth - First

```
Begin
    open := [Start];
    closed := [ ];
    while open π [ ] do
        begin
            remove leftmost state from open, call it X;
                if X is a goal then returns SUCCESS
                    else begin
                        generate children of X;
                        put X on closed;
                        discard children of X if already on open or closed;
                        put remaining children on left end of open
                    end
        end
    return FAIL
end
```

Depth - First



Depth - First

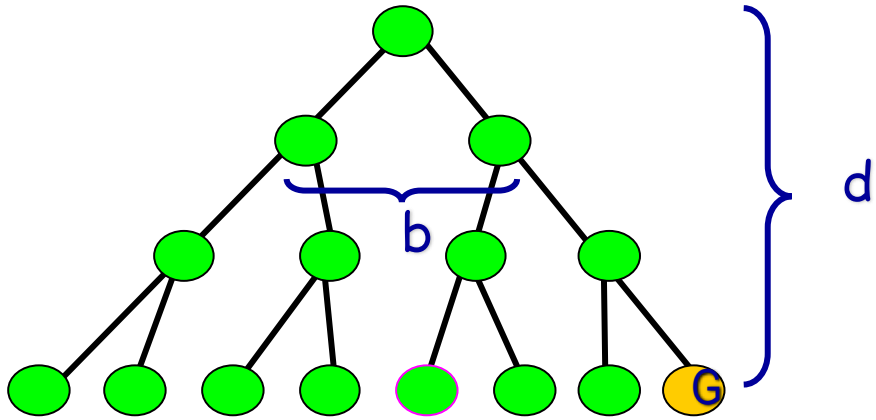


1. open = [A]; closed = [ ]
2. open = [B,C,D]; closed = [A]
3. open = [E,F,C,D]; closed = [B,A]
4. open = [K,L,F,C,D]; closed = [E,B,A]
5. open = [S,L,F,C,D]; closed = [K,E,B,A]
6. open = [L,F,C,D]; closed = [S,K,E,B,A]
7. open = [T,F,C,D]; closed = [L,S,K,E,B,A]
8. open = [F,C,D]; closed = [T,L,S,K,E,B,A]

Continua hasta encontrar U, o open = [ ].

Depth - First

- Completo
  - Si -> árboles infinitos
- Tiempo
  - En el peor de los casos, el estado final podría ser la última rama del árbol.
  - $O(b^d)$



Depth - First

- Memoria
  - ✓En general: O(d*b)

